LeetCode in C

Part I: LeetCode patterns and C templates | Part II: what makes a great programmer

1. How LeetCode works in C

You never write `main`. LeetCode gives you a function signature and calls it with many test cases. You write only that function (plus helpers). Headers like stdio, stdlib, string, stdbool, and limits are already included.

```
// Problem: Two Sum. LeetCode gives you this:
int* twoSum(int* nums, int numsSize, int target, int* returnSize);
```

Part of the signature	Meaning
`int* nums, int numsSize`	An array does not carry its length in C, so the size is passed separately.
`int* returnSize`	An **output parameter**: you must write the length of your returned array into `*returnSize`.
Return type `int*`	Return a **malloc'd** array. LeetCode frees it after checking.
`int** returnColumnSizes`	For 2D results: allocate an int array and store each row's length in it.
`char** grid, int gridSize, int* gridColSize`	2D input: gridSize = rows, gridColSize[0] = columns.
`struct ListNode*`	Linked list nodes are already defined. Do not redefine them.
`struct TreeNode*`	Tree nodes are already defined (val, left, right).

Test your solution locally first

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

/* paste your solution here */

int main(void)
{
    int nums[] = {2, 7, 11, 15};
    int size;
    int *r = twoSum(nums, 4, 9, &size);
    printf("%d %d (size %d)\n", r[0], r[1], size);
    free(r);
    return 0;
}
```

Compile with `gcc -Wall -Wextra -g -fsanitize=address,undefined file.c`. This catches most bugs before LeetCode shows you a vague runtime error.

2. Big-O: the cheat sheet

Complexity	Name	Example
O(1)	Constant	Array index, hash lookup, push on a stack
O(log n)	Logarithmic	Binary search, heap push/pop
O(n)	Linear	One loop, linked-list traversal
O(n log n)	Linearithmic	Sorting (qsort), merge sort
O(n^2)	Quadratic	Two nested loops, bubble sort
O(2^n)	Exponential	All subsets, naive Fibonacci
O(n!)	Factorial	All permutations

Use the constraints to pick the algorithm

LeetCode states the input size. A typical judge handles about 10^8 simple operations per second, so the limit tells you what is allowed.

Input size n	Target complexity	Typical techniques
n <= 10	O(n!)	Permutations, brute-force backtracking
n <= 20	O(2^n)	Subsets, bitmask, backtracking
n <= 500	O(n^3)	Triple loops, Floyd-Warshall
n <= 10^4	O(n^2)	Nested loops, simple DP
n <= 10^5	O(n log n)	Sorting, heap, binary search, divide and conquer
n <= 10^6	O(n)	Hash table, two pointers, sliding window, prefix sums
n > 10^6	O(log n) or O(1)	Math, binary search on the answer

3. A 6-step method for any problem

- **1. Understand.** Restate the problem in your own words. What are the input, output, and constraints?
- **2. Examples and edge cases.** Write 2-3 examples by hand. Think: empty input, one element, duplicates, negatives, all equal, maximum size.
- **3. Brute force.** Say the simplest solution and its complexity out loud. It is your baseline and usually correct.
- **4. Optimize.** Ask what repeated work you can remove. Can a hash table, sorting, two pointers, or a stack help? Use the pattern table below.
- **5. Code.** Write clean code with clear variable names. Allocate, set returnSize, and free temporary memory.
- **6. Test.** Walk through your examples line by line, then your edge cases. Check the complexity.

Tip: If you are stuck for 20-30 minutes, read the hint or solution, then close it and re-implement from memory the next day. Learning from solutions is normal and is how patterns stick.

4. Pattern recognition: clue to technique

If you see...	Try...	Example problems
Pair or complement lookup, duplicates, counting	Hash table / counting array	1, 217, 242, 49
Sorted array, pairs, palindromes, in-place removal	Two pointers	125, 15, 11, 26, 283
Longest / shortest *contiguous* subarray or substring	Sliding window	3, 121, 424
Range sums, product of others	Prefix sums	238, 560
Sorted input, or 'minimize the max / maximize the min'	Binary search (also on the answer)	704, 33, 875
Matching brackets, next greater element, undo	Stack (monotonic stack)	20, 155, 739
Linked list: reverse, middle, cycle, merge	Pointers: prev/curr, slow/fast, dummy node	206, 141, 21, 876
Tree: depth, paths, validity	DFS recursion	104, 226, 98, 543
Tree or grid by levels, shortest path (unweighted)	BFS with a queue	102, 200

If you see...	Try...	Example problems
All combinations, subsets, permutations	Backtracking	78, 46, 39, 79
Count ways, min/max cost, choices that repeat	Dynamic programming	70, 198, 322, 300, 1143
Top K, k-th largest, merge K sorted	Heap (priority queue)	215, 347, 703
Connections, dependencies, islands	Graph DFS/BFS, topological sort	200, 207, 133
Single number, powers of two, counting bits	Bit manipulation (XOR, shifts)	136, 191, 338, 268
Overlapping intervals	Sort by start, then sweep	56

5. Patterns with C templates

Each template is a skeleton you should be able to write from memory. The LeetCode numbers show where to practice.

5.1 Hash table (C has no built-in one)

For Two Sum (#1), a small open-addressing table is enough. The size is a power of two, larger than twice the element count.

```
#define SZ 32768                        /* power of two */
typedef struct { int key, val, used; } Entry;

static unsigned hash(int k) { return ((unsigned)k * 2654435761u) & (SZ - 1); }

int* twoSum(int* nums, int n, int target, int* returnSize)
{
    Entry *t = calloc(SZ, sizeof(Entry));
    int *res = malloc(2 * sizeof(int));
    *returnSize = 2;
    for (int i = 0; i < n; i++)
    {
        int need = target - nums[i];
        for (unsigned h = hash(need); t[h].used; h = (h + 1) & (SZ - 1))
            if (t[h].key == need)
            {
                res[0] = t[h].val; res[1] = i;
                free(t);
                return res;
            }
        unsigned h = hash(nums[i]);
        while (t[h].used) h = (h + 1) & (SZ - 1);
        t[h] = (Entry){ nums[i], i, 1 };
    }
    free(t); *returnSize = 0;
    return res;
}
```

Counting arrays are even simpler when the key range is small, such as lowercase letters or ASCII:

```
int cnt[26] = {0};
for (int i = 0; s[i]; i++) cnt[s[i] - 'a']++;     /* Valid Anagram #242 */
```

5.2 Two pointers

```
/* Valid Palindrome #125 (ignore non-alphanumeric, ignore case) */
bool isPalindrome(char *s)
{
    int l = 0, r = strlen(s) - 1;
    while (l < r)
    {
        while (l < r && !isalnum((unsigned char)s[l])) l++;
        while (l < r && !isalnum((unsigned char)s[r])) r--;
        if (tolower((unsigned char)s[l]) != tolower((unsigned char)s[r])) return false;
        l++; r--;
    }
    return true;
}

/* Two pointers on a sorted array: find a pair summing to target */
int l = 0, r = n - 1;
while (l < r)
{
    int sum = a[l] + a[r];
    if (sum == target) { /* found */ break; }
    else if (sum < target) l++;
    else r--;
```

```
}
/* 3Sum #15: sort, fix a[i], then run this on i+1..n-1; skip duplicates. */
```

5.3 Sliding window

```
/* Longest Substring Without Repeating Characters #3 */
int lengthOfLongestSubstring(char *s)
{
    int last[256];
    for (int i = 0; i < 256; i++) last[i] = -1;
    int best = 0, left = 0;
    for (int r = 0; s[r]; r++)
    {
        unsigned char c = s[r];
        if (last[c] >= left) left = last[c] + 1;   /* shrink window */
        last[c] = r;
        if (r - left + 1 > best) best = r - left + 1;
    }
    return best;
}
```

General shape: expand *right* every step, shrink *left* while the window is invalid, and update the answer each time the window is valid.

5.4 Prefix sums / running values

```
/* Product of Array Except Self #238: no division, O(n) */
int* productExceptSelf(int* nums, int n, int* returnSize)
{
    int *res = malloc(n * sizeof(int));
    *returnSize = n;
    int prefix = 1;
    for (int i = 0; i < n; i++) { res[i] = prefix; prefix *= nums[i]; }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) { res[i] *= suffix; suffix *= nums[i]; }
    return res;
}
```

```
/* Maximum Subarray #53 (Kadane) */
int maxSubArray(int* nums, int n)
{
    int cur = nums[0], best = nums[0];
    for (int i = 1; i < n; i++)
    {
        cur = (cur > 0) ? cur + nums[i] : nums[i];
        if (cur > best) best = cur;
    }
    return best;
}
```

5.5 Binary search

```
/* Classic: find target, return index or -1 (#704) */
int search(int* nums, int n, int target)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;          /* avoids overflow */
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

```
/* Binary search on the ANSWER: smallest x where ok(x) is true (e.g. #875) */
```

```
int lo = 1, hi = MAX_POSSIBLE;
while (lo < hi)
{
    int mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;                    /* mid might be the answer */
    else lo = mid + 1;
}
return lo;
```

5.6 Sorting with qsort

```
/* Safe comparator: never use (a - b), it can overflow */
int cmp(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}
qsort(nums, n, sizeof(int), cmp);

/* Intervals #56: sort an array of int[2] by start */
int cmp_iv(const void *a, const void *b)
{
    const int *x = *(const int **)a, *y = *(const int **)b;   /* int** rows */
    return (x[0] > y[0]) - (x[0] < y[0]);
}
```

5.7 Stack

```
/* Valid Parentheses #20 */
bool isValid(char *s)
{
    int n = strlen(s), top = 0;
    char *st = malloc(n + 1);
    bool ok = true;
    for (int i = 0; i < n && ok; i++)
    {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') st[top++] = c;
        else if (top == 0) ok = false;
        else
        {
            char o = st[--top];
            if ((c == ')' && o != '(') || (c == ']' && o != '[')
                || (c == '}' && o != '{'))
                ok = false;
        }
    }
    ok = ok && top == 0;
    free(st);
    return ok;
}
```

Monotonic stack (Daily Temperatures #739): keep a stack of indices whose values are decreasing. When a new value is larger than the top, pop and record the answer for the popped index.

5.8 Linked lists

```
/* Reverse #206 */
struct ListNode* reverseList(struct ListNode* head)
{
    struct ListNode *prev = NULL;
    while (head)
    {
        struct ListNode *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
```

```
    }
    return prev;
}

/* Merge two sorted lists #21 with a dummy node */
struct ListNode* mergeTwoLists(struct ListNode* a, struct ListNode* b)
{
    struct ListNode dummy, *tail = &dummy;
    dummy.next = NULL;
    while (a && b)
    {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else                  { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

/* Cycle detection #141 (Floyd): slow moves 1, fast moves 2 */
bool hasCycle(struct ListNode *head)
{
    struct ListNode *slow = head, *fast = head;
    while (fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Tip: Use a dummy node whenever the head can change (merge, remove node). Slow/fast pointers also find the middle (#876) and the Nth node from the end (#19, give the fast pointer an N-step head start).

5.9 Trees: DFS

```
/* Max Depth #104 */
int maxDepth(struct TreeNode* root)
{
    if (!root) return 0;
    int l = maxDepth(root->left), r = maxDepth(root->right);
    return 1 + (l > r ? l : r);
}

/* Invert #226 */
struct TreeNode* invertTree(struct TreeNode* root)
{
    if (!root) return NULL;
    struct TreeNode *tmp = root->left;
    root->left = invertTree(root->right);
    root->right = invertTree(tmp);
    return root;
}

/* Validate BST #98: pass the allowed range down (use long long bounds) */
static bool check(struct TreeNode *n, long long lo, long long hi)
{
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return check(n->left, lo, n->val) && check(n->right, n->val, hi);
}
bool isValidBST(struct TreeNode* root) { return check(root, LLONG_MIN, LLONG_MAX); }
```

Recursion recipe for trees: (1) base case: `NULL`; (2) solve left and right; (3) combine their answers with the current node. Think of what each call must *return*.

5.10 BFS (level order)

```
/* Core loop; q is an array queue of pointers, N = max number of nodes */
struct TreeNode **q = malloc(N * sizeof(*q));
int head = 0, tail = 0;
q[tail++] = root;
while (head < tail)
{
    int level = tail - head;               /* nodes on this level */
    for (int i = 0; i < level; i++)
    {
        struct TreeNode *node = q[head++];
        /* visit node */
        if (node->left)  q[tail++] = node->left;
        if (node->right) q[tail++] = node->right;
    }
}
free(q);
```

5.11 Grid DFS (islands)

```
/* Number of Islands #200 */
static void dfs(char **g, int r, int c, int R, int C)
{
    if (r < 0 || c < 0 || r >= R || c >= C || g[r][c] != '1') return;
    g[r][c] = '0';                         /* mark visited */
    dfs(g, r + 1, c, R, C); dfs(g, r - 1, c, R, C);
    dfs(g, r, c + 1, R, C); dfs(g, r, c - 1, R, C);
}
int numIslands(char** grid, int gridSize, int* gridColSize)
{
    int R = gridSize, C = gridColSize[0], count = 0;
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++)
            if (grid[r][c] == '1') { count++; dfs(grid, r, c, R, C); }
    return count;
}
```

5.12 Backtracking

Template: choose, explore, un-choose. Subsets of an array with n <= 10 can be done with bitmasks, which avoids complicated memory handling in C:

```
/* Subsets #78 */
int** subsets(int* nums, int n, int* returnSize, int** returnColumnSizes)
{
    int total = 1 << n;
    int **res = malloc(total * sizeof(int *));
    *returnColumnSizes = malloc(total * sizeof(int));
    *returnSize = total;
    for (int mask = 0; mask < total; mask++)
    {
        int cnt = __builtin_popcount(mask);
        res[mask] = malloc((cnt ? cnt : 1) * sizeof(int));
        (*returnColumnSizes)[mask] = cnt;
        int k = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) res[mask][k++] = nums[i];
    }
    return res;
}

/* General backtracking skeleton */
void solve(state, choices)
{
    if (is_complete(state)) { record(state); return; }
    for (each choice in choices)
    {
```

```
        if (!valid(choice)) continue;
        apply(choice);              /* choose   */
        solve(state, choices);      /* explore  */
        undo(choice);               /* un-choose */
    }
}
```

5.13 Dynamic programming

Steps: (1) define what `dp[i]` means; (2) write the recurrence; (3) set base cases; (4) choose the fill order; (5) read the answer. Start recursive with memo if the recurrence is easier to see, then convert to a table.

```
/* Climbing Stairs #70: dp[i] = dp[i-1] + dp[i-2] (space O(1)) */
int climbStairs(int n)
{
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) { int t = a + b; a = b; b = t; }
    return b;
}

/* House Robber #198: dp[i] = max(dp[i-1], dp[i-2] + nums[i]) */
int rob(int* nums, int n)
{
    int prev2 = 0, prev1 = 0;
    for (int i = 0; i < n; i++)
    {
        int cur = prev1 > prev2 + nums[i] ? prev1 : prev2 + nums[i];
        prev2 = prev1; prev1 = cur;
    }
    return prev1;
}

/* Coin Change #322: dp[a] = min coins to make amount a */
int coinChange(int* coins, int n, int amount)
{
    int *dp = malloc((amount + 1) * sizeof(int));
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
    {
        dp[a] = amount + 1;                     /* "infinity" */
        for (int i = 0; i < n; i++)
            if (coins[i] <= a && dp[a - coins[i]] + 1 < dp[a])
                dp[a] = dp[a - coins[i]] + 1;
    }
    int res = dp[amount] > amount ? -1 : dp[amount];
    free(dp);
    return res;
}
```

5.14 Heap (priority queue)

C has no heap in the standard library, so write a small one. This is a min-heap of ints. For 'K largest', keep a min-heap of size K.

```
typedef struct { int *a; int size; } Heap;
static void swp(int *x, int *y) { int t = *x; *x = *y; *y = t; }

void push(Heap *h, int v)
{
    int i = h->size++;
    h->a[i] = v;
    while (i > 0 && h->a[(i - 1) / 2] > h->a[i])   /* sift up */
    {
        swp(&h->a[(i - 1) / 2], &h->a[i]);
        i = (i - 1) / 2;
    }
}
int pop(Heap *h)
```

```
{
    int top = h->a[0];
    h->a[0] = h->a[--h->size];
    int i = 0;
    for (;;)                                        /* sift down */
    {
        int l = 2 * i + 1, r = l + 1, m = i;
        if (l < h->size && h->a[l] < h->a[m]) m = l;
        if (r < h->size && h->a[r] < h->a[m]) m = r;
        if (m == i) break;
        swp(&h->a[m], &h->a[i]);
        i = m;
    }
    return top;
}
```

5.15 Bit manipulation

```
/* Single Number #136: pairs cancel out under XOR */
int singleNumber(int* nums, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) x ^= nums[i];
    return x;
}

/* Number of 1 Bits #191: x &= x - 1 clears the lowest set bit */
int hammingWeight(uint32_t n) { int c = 0; while (n) { n &= n - 1; c++; } return c; }

/* Counting Bits #338: bits[i] = bits[i >> 1] + (i & 1) */
/* Missing Number #268: XOR all indices 0..n with all values; or n(n+1)/2 - sum */
```

6. C-specific traps on LeetCode

Trap	What happens	Fix
Forgetting `*returnSize`	Wrong answer or crash	Set it on **every** return path, including empty results
Returning a local array	Dangling pointer, garbage values	malloc the result
Static / global variables	They persist between test cases and carry old data	Reset them at the start of the function, or avoid them
Integer overflow	Wrong sums / products	Use `long long`; compare to INT_MAX / LLONG_MAX
`abs(INT_MIN)`	Undefined behavior	Use long long first
`mid = (lo + hi) / 2`	Overflow for large indexes	`lo + (hi - lo) / 2`
Missing null terminator	Garbage after returned strings	Allocate len + 1 and set `'\0'`
`strlen` in a loop condition	Turns O(n) into O(n^2)	Compute it once before the loop
`size_t i` counting down	`i >= 0` is always true	Use int, or loop with `i-- > 0`
`sizeof(arr)` on a parameter	Gives the pointer size, not the array size	Use the size parameter you were given
Deep recursion (n = 10^5)	Stack overflow	Convert to an iterative solution with an explicit stack
Modifying input strings	Fails if the input is a literal or must be preserved	Copy first when unsure
Memory leaks in temporary buffers	Usually passes, but is bad habit	Free what you allocate, except the returned data

7. Roadmap: problems by topic

Difficulty: E = Easy, M = Medium. Do them in this order. Finish one topic before moving on, and re-solve any you needed help with after a few days.

Arrays and hashing

#	Problem	Diff	Key idea
217	Contains Duplicate	E	Hash set or sort
242	Valid Anagram	E	Counting array of 26
1	Two Sum	E	Hash map: complement lookup
49	Group Anagrams	M	Sort each word as the key, or count signature
347	Top K Frequent Elements	M	Count, then heap or bucket by frequency
238	Product of Array Except Self	M	Prefix and suffix products
128	Longest Consecutive Sequence	M	Hash set; only start at sequence beginnings
169	Majority Element	E	Boyer-Moore voting
283	Move Zeroes	E	Write pointer

Two pointers

#	Problem	Diff	Key idea
125	Valid Palindrome	E	Pointers from both ends
26	Remove Duplicates from Sorted Array	E	Read and write pointers
27	Remove Element	E	Read and write pointers
88	Merge Sorted Array	E	Merge from the back
11	Container With Most Water	M	Move the shorter side inward
15	3Sum	M	Sort, fix one, two pointers, skip duplicates

Sliding window

#	Problem	Diff	Key idea
121	Best Time to Buy and Sell Stock	E	Track min so far
3	Longest Substring Without Repeating Characters	M	Last-seen index per char
424	Longest Repeating Character Replacement	M	Window valid if len - maxCount <= k

Stack

#	Problem	Diff	Key idea
20	Valid Parentheses	E	Stack of openers
155	Min Stack	M	Store (value, min so far) pairs
150	Evaluate Reverse Polish Notation	M	Stack of operands
739	Daily Temperatures	M	Monotonic decreasing stack of indices

Binary search

#	Problem	Diff	Key idea
704	Binary Search	E	Template from section 5.5

#	Problem	Diff	Key idea
69	Sqrt(x)	E	Binary search on the answer, use long long
74	Search a 2D Matrix	M	Treat as one sorted array of m*n
153	Find Minimum in Rotated Sorted Array	M	Compare mid with right end
33	Search in Rotated Sorted Array	M	Find which half is sorted
875	Koko Eating Bananas	M	Binary search on speed

Linked lists

#	Problem	Diff	Key idea
206	Reverse Linked List	E	prev / curr / next
21	Merge Two Sorted Lists	E	Dummy node
141	Linked List Cycle	E	Slow and fast pointers
876	Middle of the Linked List	E	Slow and fast pointers
19	Remove Nth Node From End of List	M	Dummy node plus gap of N
2	Add Two Numbers	M	Digit by digit with carry
143	Reorder List	M	Find middle, reverse second half, merge

Trees

#	Problem	Diff	Key idea
104	Maximum Depth of Binary Tree	E	1 + max(left, right)
226	Invert Binary Tree	E	Swap children recursively
100	Same Tree	E	Compare node and both subtrees
110	Balanced Binary Tree	E	Return height, or -1 if unbalanced
543	Diameter of Binary Tree	E	Height plus a global best
102	Binary Tree Level Order Traversal	M	BFS with level size
98	Validate Binary Search Tree	M	Pass min and max bounds down
235	Lowest Common Ancestor of a BST	M	Walk down using value comparisons

Backtracking

#	Problem	Diff	Key idea
78	Subsets	M	Bitmask or choose/skip
46	Permutations	M	Swap-based or used[] array
39	Combination Sum	M	Reuse allowed: recurse at same index
79	Word Search	M	Grid DFS, mark and unmark visited

Heap

#	Problem	Diff	Key idea
703	Kth Largest Element in a Stream	E	Min-heap of size K
215	Kth Largest Element in an Array	M	Min-heap of size K, or quickselect

Graphs

#	Problem	Diff	Key idea
200	Number of Islands	M	Grid DFS or BFS
695	Max Area of Island	M	DFS returning area
133	Clone Graph	M	DFS with an old-to-new map
207	Course Schedule	M	Cycle detection / topological sort

Dynamic programming

#	Problem	Diff	Key idea
70	Climbing Stairs	E	Fibonacci
198	House Robber	M	Take or skip
62	Unique Paths	M	dp[r][c] = dp[r-1][c] + dp[r][c-1]
322	Coin Change	M	Unbounded knapsack
300	Longest Increasing Subsequence	M	O(n^2) dp first, then O(n log n)
139	Word Break	M	dp[i] = some word ends at i and dp[j] is true
1143	Longest Common Subsequence	M	2D dp over both strings
53	Maximum Subarray	M	Kadane

Bit manipulation and math

#	Problem	Diff	Key idea
136	Single Number	E	XOR everything
191	Number of 1 Bits	E	n &= n - 1
338	Counting Bits	E	bits[i] = bits[i/2] + (i%2)
190	Reverse Bits	E	Shift out and in, 32 times
268	Missing Number	E	XOR or sum formula
9	Palindrome Number	E	Reverse half the number
13	Roman to Integer	E	Subtract if next symbol is larger
66	Plus One	E	Carry from the end
67	Add Binary	E	Carry from the end of both strings

Arrays and matrices (extra)

#	Problem	Diff	Key idea
14	Longest Common Prefix	E	Compare column by column
28	Find the Index of the First Occurrence in a String	E	Naive scan
56	Merge Intervals	M	Sort by start, then merge
189	Rotate Array	M	Triple reverse
48	Rotate Image	M	Transpose, then reverse rows
54	Spiral Matrix	M	Four shrinking boundaries
73	Set Matrix Zeroes	M	Use first row/col as markers

8. An 8-week study plan

About 1 hour a day, 5-6 days a week. Aim for 3-5 problems per week per topic early on, quality over quantity.

Week	Focus	Goal
1	C fundamentals, arrays, hashing, counting	Comfortable with returnSize, malloc, and building a hash table
2	Two pointers, sliding window, prefix sums	Recognize these from the problem statement
3	Stack, binary search	Write binary search templates from memory
4	Linked lists	Reverse, merge, cycle, and dummy-node patterns without bugs
5	Trees (DFS and BFS)	Think recursively: base case plus combine
6	Backtracking and heap	Choose / explore / un-choose; write a heap
7	Graphs and dynamic programming	Grid DFS/BFS; define dp state before coding
8	Mixed review, timed practice	Solve mediums in 30 minutes; redo past misses

9. Habits that make you improve

- **Keep a mistake log.** For each problem you miss, write the pattern, the trick, and why you missed it.
- **Re-solve after 3 days and again after 2 weeks.** Spaced repetition is what moves a pattern into long-term memory.
- **Say your approach before coding.** This is exactly what interviews test.
- **Always state time and space complexity** of your solution.
- **Do timed sessions** (30-45 minutes per problem) once you are past week 4.
- **Compare solutions.** After solving, read a top solution and note what was cleaner or faster.
- **Do not memorize code, memorize patterns.** If you know the why, the code follows.
- **Be consistent.** One problem a day beats ten on Sunday.

Quick checklist before you submit

- Did I handle the empty / one-element / all-equal cases?
- Did I set `*returnSize` (and returnColumnSizes if needed) on every return path?
- Is every returned array or string malloc'd and null-terminated?
- Could any sum or product overflow an int?
- Do my loop bounds avoid reading outside the array?
- Did I free temporary memory I allocated?
- Do global or static variables carry state between test cases?

Part II

Beyond LeetCode: what you need to know to be great at programming

10. The big picture: six pillars

LeetCode trains one skill: solving small, well-defined puzzles. Great programmers are also strong in five other areas. Weakness in any pillar shows up quickly in real work.

Pillar	What it means	How you build it
1. Fundamentals	Language mastery, types, memory, pointers, control flow	Write lots of C; read K&R; do the checklist tasks
2. CS theory	Data structures, algorithms, complexity, discrete math	LeetCode, CLRS, writing structures from scratch
3. Systems	How CPU, memory, OS, compiler, and network actually work	CS:APP, OSTEP, Beej's Guides, build a shell / allocator / server
4. Engineering	Clean design, testing, debugging, version control, review	Real projects, git, tests, code review
5. Tools	Linux CLI, editor, debugger, profiler, build systems	Daily use; automate everything you repeat
6. Soft skills	Communication, learning speed, estimation, teamwork	Write docs, explain code, collaborate, read other people's code

Tip: Depth beats breadth early. Get really good at one language (C is an excellent choice because it exposes how computers work), then add others quickly because the concepts transfer.

11. Core computer science knowledge

11.1 Data structures you must know cold

Structure	Operations and cost	Use it when
Array / dynamic array	Index O(1); append amortized O(1); insert middle O(n)	Default choice; cache-friendly
Linked list	Insert/delete at known node O(1); search O(n)	Frequent insert/remove, no random access
Stack / Queue / Deque	Push, pop, peek O(1)	Undo, parsing, BFS, scheduling
Hash table	Average O(1) lookup; worst O(n)	Fast lookup, counting, deduplication
Binary search tree / balanced tree	O(log n) if balanced	Ordered data, range queries
Heap	Push/pop O(log n); peek O(1)	Priority queue, top-K, scheduling
Graph (list / matrix)	BFS/DFS O(V+E)	Networks, dependencies, maps
Trie	O(length of key)	Prefix search, autocomplete
Union-Find	Nearly O(1) per op	Connected components, Kruskal

11.2 Algorithms to know

- **Sorting:** merge sort, quicksort, heap sort, counting sort. Know their complexity and stability.
- **Searching:** binary search, BFS, DFS, Dijkstra, A* (idea), topological sort.
- **Paradigms:** brute force, divide and conquer, greedy, dynamic programming, backtracking.
- **String algorithms:** two pointers, KMP (idea), hashing, tries.
- **Math:** GCD, primes and sieve, modular arithmetic, fast exponentiation, combinatorics basics.

- **Analysis:** Big-O, amortized analysis, space vs time trade-offs.

11.3 Computer architecture (why C behaves the way it does)

- **Memory hierarchy:** registers, L1/L2/L3 cache, RAM, disk. A cache miss costs 100x a hit. This is why arrays often beat linked lists in practice.
- **Data representation:** two's complement, IEEE-754 floats (why 0.1 + 0.2 != 0.3), endianness, alignment and padding.
- **Stack frames:** how function calls, arguments, return addresses, and local variables are laid out; why stack overflow and buffer overflows happen.
- **Instruction basics:** be able to read simple x86-64 or ARM assembly (`gcc -S`).
- **Pipelining and branch prediction:** why predictable code runs faster.

11.4 Operating systems

- **Processes vs threads:** address spaces, context switching, `fork`, `exec`.
- **Virtual memory:** pages, page faults, `mmap`, how `malloc` gets memory from the kernel (`brk` / `mmap`).
- **File descriptors:** stdin/stdout/stderr = 0/1/2, pipes, redirection, sockets are files too.
- **System calls:** `read`, `write`, `open`, `close`, `wait`, `kill`; the user/kernel boundary.
- **Concurrency:** race conditions, mutexes, deadlock (and the four conditions), semaphores, atomics, producer/consumer.
- **Scheduling and signals:** how the OS shares the CPU; how signals interrupt your program.

11.5 Networking

- The layers: link, IP, TCP/UDP, application. What TCP guarantees that UDP doesn't.
- Sockets API: `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`.
- HTTP basics (methods, status codes, headers), DNS, TLS at a conceptual level.
- Why you must handle partial reads/writes and why `recv` may return fewer bytes than you asked for.

11.6 Databases

- SQL: SELECT, JOIN, GROUP BY, indexes, and reading a query plan.
- Transactions and ACID; why indexes speed reads but slow writes.
- Relational vs key-value vs document stores: when each fits.

11.7 How a C program becomes a process

```
source.c --preprocess--> expanded .i --compile--> assembly .s --assemble--> object .o
                                                                              |
   libraries (.a / .so)  ---------------------------- link --------+--> executable
                                                                              |
                         loader maps it into memory --> process runs main()

gcc -E file.c   # preprocessor only     gcc -S file.c   # to assembly
gcc -c file.c   # to object file        nm / objdump -d # inspect symbols and code
```

12. Writing good code

Anyone can make code work. Great programmers make code that is **correct, readable, and easy to change**. Code is read far more often than it is written.

12.1 Principles

Principle	Meaning
KISS	Keep it simple. The simplest solution that works is usually best.
DRY	Don't repeat yourself. Duplicated logic means duplicated bugs.
YAGNI	Don't build features you don't need yet.
Single responsibility	Each function does one thing, and its name says what.
Separation of concerns	Keep parsing, logic, and I/O in separate functions or modules.
Least surprise	Functions behave the way their names and signatures suggest.
Fail fast	Detect bad input early and report it clearly instead of continuing.
Premature optimization	Make it correct first, measure, then optimize the real bottleneck.

12.2 Practical rules

- **Names:** `count_active_users` beats `cnt`; avoid single letters except for loop indexes.
- **Small functions:** if it doesn't fit on a screen, split it. 5-25 lines is a good target.
- **Few parameters:** more than 4 is a smell; group them into a struct.
- **Comments explain why, not what.** If a comment repeats the code, delete it or rename things.
- **No magic numbers:** use `#define`, `enum`, or `const` with a name.
- **Avoid global mutable state.** Pass data explicitly.
- **Handle every error.** Check return values, and clean up on every exit path.
- **Be consistent** in formatting and naming; adopt a style guide or `clang-format`.

12.3 Good C design

- **Ownership:** for every pointer, know who allocates and who frees. Document it.
- **Use** `const` for pointers you won't modify: `size_t ft_strlen(const char *s)`.
- **Use** `static` for functions private to a file.
- **Return errors, not crashes.** Report failure by return value; don't call `exit` inside library code.
- **Hide implementation** with opaque types so callers can't depend on your internals.

```
/* stack.h: callers see only this */
typedef struct Stack Stack;              /* fields are hidden in stack.c */

Stack *stack_new(void);
int    stack_push(Stack *s, int v);      /* 0 = ok, -1 = out of memory      */
int    stack_pop(Stack *s, int *out);    /* 0 = ok, -1 = empty              */
void   stack_free(Stack *s);             /* accepts NULL, like free()       */
```

12.4 Undefined behavior: the silent killer in C

Undefined behavior (UB) means the standard places no requirements on what happens. The program may crash, work today and break tomorrow, or be 'optimized' into something wrong. Learn this list and use `-fsanitize=undefined,address`.

UB	Example
Out-of-bounds access	`a[10]` on an array of 10
Signed integer overflow	`INT_MAX + 1` (unsigned overflow is defined and wraps)
NULL / dangling pointer dereference	Use after free, returning the address of a local
Uninitialized read	Reading a local variable or malloc memory before writing
Double free / invalid free	`free(p); free(p);`
Bad shifts	Shifting by >= width or by a negative amount
Division by zero	`x / 0`, also `INT_MIN / -1`

UB	Example
Strict aliasing violation	Reading a float through an int pointer
Modifying a string literal	`char *s = "hi"; s[0] = 'H';`
Unsequenced modification	`i = i++ + ++i;`
Missing return in non-void function	Falling off the end of a function that returns a value

13. Debugging: the skill that separates good from great

13.1 The method

- **1. Reproduce.** Find the smallest input that triggers the bug, and make it deterministic.
- **2. Observe, don't guess.** Look at actual values with a debugger or prints before forming theories.
- **3. Form a hypothesis** that explains *all* the symptoms.
- **4. Test one change at a time.** If you change three things, you learn nothing.
- **5. Bisect.** Halve the search space: comment out half the code, or use `git bisect` to find the commit that introduced the bug.
- **6. Fix the cause, not the symptom,** then add a test so it never returns.
- **Rubber duck:** explain the code line by line, out loud. You will often find the bug mid-sentence.

13.2 gdb cheat sheet

Command	What it does
`gcc -g -O0 file.c`	Build with debug symbols and no optimization
`gdb ./a.out`	Start the debugger
`break main / break file.c:42`	Set a breakpoint
`run [args]`	Start the program
`next / step / finish`	Next line / step into a call / run until the function returns
`print var / print *ptr / print arr[0]@5`	Inspect values (the last prints 5 array elements)
`display var`	Show a value after every stop
`backtrace (bt) / frame N`	See the call stack / jump to a frame
`watch var`	Stop whenever a variable changes
`continue`	Run to the next breakpoint
`x/16xb ptr`	Examine 16 bytes of memory in hex
`info locals / info args`	Show local variables and arguments

13.3 Other tools

- **AddressSanitizer / UBSan:** `-fsanitize=address,undefined`. Catches most memory bugs instantly.
- **Valgrind:** `valgrind --leak-check=full ./a.out`. Leaks and invalid reads.
- **strace:** shows every system call a program makes. Great for 'why can't it open my file?'.
- **ltrace, ldd, nm, objdump:** library calls, linked libraries, symbols, disassembly.
- **Core dumps:** `ulimit -c unlimited`, then `gdb ./a.out core` to inspect a crash after the fact.
- **Static analysis:** `clang-tidy`, `cppcheck`, and `-Wall -Wextra -Wpedantic -Wshadow -Wconversion`.

Tip: Treat compiler warnings as errors (`-Werror`). Most of the bugs beginners spend hours on were warned about at compile time.

14. Tools every programmer should master

14.1 Linux command line

Task	Commands
Navigate and inspect	ls -la, cd, pwd, tree, file, stat, du -sh, df -h
View and search files	cat, less, head, tail -f, grep -rn, find . -name, wc -l
Text processing	sed, awk, sort, uniq -c, cut, tr, xargs
Pipes and redirection	cmd1 | cmd2, > file, >> file, 2>&1, < file
Processes	ps aux, top / htop, kill, jobs, bg, fg
Permissions	chmod, chown, umask
Networking	curl, wget, ping, ss -tulpn, nc, dig, ssh, scp
Archives and misc	tar -czf / -xzf, diff, man, which, alias, history

Learn **Bash scripting** basics (variables, loops, conditions, functions) and a terminal editor (**vim** or **nano**) well enough to work on any server. Pick a main editor (VS Code, Neovim, or similar) and learn its shortcuts, debugger integration, and search tools deeply.

14.2 Git: non-negotiable

```
git init / git clone <url>
git status                      # what changed?
git add -p                      # stage changes interactively, hunk by hunk
git commit -m "Fix off-by-one in ring buffer"
git log --oneline --graph --all
git diff / git diff --staged
git switch -c feature/parser    # new branch
git merge feature/parser        # or: git rebase main
git stash / git stash pop       # shelve work temporarily
git bisect start                # binary-search for the commit that broke things
git restore <file>              # discard local changes to a file
```

- Commit small, focused changes with messages that say **what and why**.
- Never commit secrets (keys, passwords) or build outputs; use `.gitignore`.
- Learn the pull-request workflow, how to review others' code, and how to resolve merge conflicts calmly.
- Know how to undo mistakes: `git reflog` can recover almost anything you committed.

14.3 Build systems

```
CC      = gcc
CFLAGS  = -Wall -Wextra -Werror -g
SRC     = main.c parser.c util.c
OBJ     = $(SRC:.c=.o)
NAME    = app

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(CFLAGS) -o $@ $(OBJ)      # (recipe lines must start with a TAB)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ)
```

```
fclean: clean
    rm -f $(NAME)

re: fclean all
.PHONY: all clean fclean re
```

After Make, learn **CMake** (the de-facto standard for larger C/C++ projects), and how to link libraries, set include paths, and build in Debug vs Release modes.

14.4 Profiling and performance

- **Measure first.** Use `time`, `perf`, `gprof`, or `valgrind --tool=callgrind` to find the real hotspot.
- Algorithm choice (O(n log n) vs O(n^2)) beats micro-optimization.
- Then look at memory access patterns: contiguous arrays and fewer allocations are often the biggest wins.
- Compile with `-O2` for benchmarks; never benchmark with `-O0`.

15. Testing and code quality

- **Test the edges,** not just the happy path: empty, one element, maximum size, negative, duplicates, NULL, invalid input.
- **Write a test before fixing a bug.** If it didn't fail first, it doesn't prove the fix.
- **Keep tests fast and independent,** so you run them constantly.
- **Test behavior, not implementation,** so refactoring doesn't break tests.
- **Compare against a reference:** your `ft_strlen` vs the real `strlen` on random inputs; a brute-force solution vs your optimized one.
- **Fuzzing** (AFL++, libFuzzer) finds crashes in parsers by feeding random input; try it on any code that handles untrusted data.

A tiny unit-test harness in C

```
#include <stdio.h>
static int run = 0, failed = 0;
#define CHECK(cond) do { run++; if (!(cond)) { failed++; \
    fprintf(stderr, "FAIL %s:%d: %s\n", __FILE__, __LINE__, #cond); } } while (0)

int main(void)
{
    CHECK(ft_strlen("") == 0);
    CHECK(ft_strlen("abc") == 3);
    CHECK(ft_atoi("  -42") == -42);
    printf("%d/%d passed\n", run - failed, run);
    return failed != 0;                /* non-zero exit code = CI fails */
}
```

For bigger projects, look at established C test frameworks such as Unity, Check, or CMocka, and run the tests automatically on every push with **continuous integration** (GitHub Actions or similar).

Code review

- As an author: keep changes small, explain the why, and respond to feedback without defensiveness.
- As a reviewer: check correctness, edge cases, error handling, naming, and tests, in that order. Be kind and specific.
- Reading good open-source code and reviews is one of the fastest ways to level up.

16. Other languages worth learning (and why)

Language	Why learn it
C++	Same machine-level thinking plus RAII, templates, and the STL; huge in games, finance, and systems.
Rust	Memory safety without a garbage collector; makes the ownership ideas you learned in C explicit and checked.
Python	Fast scripting, automation, data work, quick prototypes, and a great LeetCode language for interviews.
JavaScript / TypeScript	The language of the web; necessary for front-end and common on the back end.
Go / Java / C#	Common in back-end services and enterprise software; teach concurrency and garbage-collected design.
SQL	Nearly every real application talks to a database.
Bash	Automation and glue for your daily tools.
Assembly (basic)	Reading it demystifies the compiler, the stack, and performance.

Learn **paradigms**, not just syntax: procedural, object-oriented, functional, and concurrent. Each language you add teaches you new ways to think, and makes you better in the others.

17. Project ladder: build things

Projects are where knowledge becomes skill. Each rung below forces you to use many topics at once. Put the finished ones on GitHub with a clear README.

Level	Projects
Beginner	Number guessing game, to-do list CLI, calculator, tic-tac-toe, text-adventure
Intermediate	Your own `libft` (strlen, memcpy, printf, split), dynamic array and hash table library, `wc` / `grep` / `cat` clones, JSON parser
Advanced	A shell, a memory allocator, an HTTP server, a key-value store, a text editor, a chat server with sockets
Hero	A small interpreter or VM, a compiler for a tiny language, a database engine, a ray tracer, a tiny OS kernel, a multi-threaded web server

- Choose projects slightly **above** your current ability.
- Add features, write tests, profile it, and fix bugs; the second half of a project teaches the most.
- Contribute to an open-source project: start with documentation, tests, or 'good first issue' tickets.

18. How to learn faster

- **Build, don't just watch.** Tutorials feel like progress but build little skill. After any tutorial, rebuild the thing without looking, then change it.
- **Deliberate practice.** Work just beyond your comfort zone, get feedback, and target your weak spots.
- **Read code.** Study small, well-written C projects: Redis, SQLite, Lua, and the early Git source are famous examples. Ask 'why did they do it this way?'
- **Teach.** Explaining a concept in writing or to a friend exposes gaps in your understanding.
- **Spaced repetition.** Revisit topics and re-solve problems after days and weeks.
- **Learn from errors.** Read every error message completely and slowly; the answer is often in the first line.
- **Read the documentation.** `man` pages, the C standard library docs, and cppreference are primary sources.
- **Sleep and breaks help.** Hard bugs often solve themselves after a walk.

- **Be patient with confusion.** Pointers, recursion, and concurrency each take weeks to click. That is normal, not a sign you can't do it.

19. Books and resources

Resource	Why
The C Programming Language (K&R)	The classic, short and precise. Do the exercises.
C Programming: A Modern Approach (K. N. King)	A thorough, modern C textbook with many exercises.
Computer Systems: A Programmer's Perspective (CS:APP)	The best book on how hardware, OS, and C fit together.
Operating Systems: Three Easy Pieces (OSTEP)	Free online; clear explanations of processes, memory, and concurrency.
Introduction to Algorithms (CLRS)	The reference for algorithms; use it for depth after the basics.
Cracking the Coding Interview / NeetCode roadmap	Interview preparation and curated problem lists.
The Pragmatic Programmer	Habits and mindset of effective engineers.
Code Complete (McConnell)	Practical software construction.
Beej's Guide to C and *Beej's Guide to Network Programming*	Free, friendly, and very practical.
The Missing Semester of Your CS Education (MIT)	Free course on the shell, git, debugging, and other tools.
Harvard CS50	Free, excellent introductory course built around C.
nand2tetris	Build a computer from logic gates up to a compiler and OS.
cppreference.com, `man` pages	Look up exact behavior of library functions.

20. Interviews and career

- **Coding interviews:** practice the patterns in Part I, and **talk while you solve**: clarify, give the brute force, optimize, code, test, state complexity.
- **System design (mid/senior):** learn the building blocks: load balancers, caches, databases and indexing, queues, replication, sharding, consistency trade-offs.
- **Behavioral questions:** prepare 4-5 stories (a conflict, a failure, a bug you tracked down, a project you led) using the STAR format: Situation, Task, Action, Result.
- **Portfolio:** two or three polished projects with READMEs and tests are worth more than ten half-finished ones.
- **Communication:** write clear commit messages, docs, and design notes. Being easy to work with matters as much as raw skill.
- **Estimation:** break work into small pieces, estimate each, and add buffer for the unknown.
- **Keep learning:** technology changes; fundamentals (data structures, systems, debugging, communication) hold their value for decades.

21. A 12-month path

Months	Focus	Milestone
1-2	C basics, pointers, memory, strings, the checklist Levels 0-4	Write `libft`-style functions; zero leaks under Valgrind
3-4	Data structures from scratch, files, Makefiles, git, gdb	Dynamic array, linked list, hash table, BST library with tests

Months	Focus	Milestone
5-6	Algorithms and LeetCode (Part I roadmap), Linux CLI, Bash	60-80 problems solved; patterns recognized fast
7-8	Systems: processes, signals, pipes, threads, sockets	Build a shell and a TCP server
9-10	Architecture and OS theory (CS:APP / OSTEP), profiling, testing, CI	Allocator or key-value store with benchmarks
11	A second language (Python plus either C++ or Rust)	Rewrite one project in the new language
12	Capstone project, open-source contribution, mock interviews	Portfolio on GitHub; confident in interviews

22. Self-assessment: can you do these without help?

- Explain the difference between stack and heap, and what happens on `malloc` and `free`.
- Write a linked list, hash table, and binary search from memory, with correct memory handling.
- Read a Valgrind or AddressSanitizer report and fix the bug it points to.
- Step through a crash in gdb and read a backtrace.
- Explain why `INT_MAX + 1` is undefined and `UINT_MAX + 1` is not.
- Explain what a process, thread, and file descriptor are, and why a race condition happens.
- Write a Makefile, use git branches, and resolve a merge conflict.
- Design the API (header file) for a small module with clear ownership and error handling.
- State the time and space complexity of any code you write.
- Explain a technical idea to a beginner in plain words.

Habits of great programmers

- They **read a lot of code** and write a lot of code.
- They **stay curious** about how things work underneath.
- They **keep things simple** and delete code happily.
- They **test, measure, and verify** instead of assuming.
- They **ask good questions**: what did I expect, what happened, and what have I tried?
- They **stay humble**: everyone writes bugs, and everyone keeps learning.
- They **show up consistently.** Years of steady practice beat short bursts of intensity.